Cairnly Assistant – Project 1 Summary

Group A: Péter Palotai, András Biró, Mariam Makhniashvili, Bálint Biricz

1. Business problem and value

Every day, an online shop hears the same questions: “Is shipping free?”, “Can I return this?”, “Where is my order?”. The answers already sit in the help centre and the order system, yet support staff type them repeatedly. A customer who writes on a Saturday evening waits until Monday, because agents work only Monday to Friday, 08:00–18:00 CET.

Our example is Cairnly Outdoor, a fictional EU shop for camping and hiking gear. Its chatbot, Cairnly Assistant, has three goals: answer questions at any hour using only the shop’s own knowledge, label each message by category and urgency, and stay safe by admitting what it does not know, never inventing policies and never handling card data. By automating routine enquiries, the assistant lowers the cost per contact, answers day or night and frees human agents for complex cases, which the category and urgency labels could route to them.

2. Data and knowledge sources

Both sources are simulated and written for this project, so the ground truth is known, and no real or personal data is used. They are stored in an SQLite database (`data/cairnly.db`, built by `data/create_db.py`) that the chatbot opens read-only.

Help centre: 25 short documents (37–97 words, 1,605 words in total) across shipping, orders, returns, warranty, payment, pricing, loyalty, account, stores, contact and safety, plus four product pages and a size guide. Policies follow common EU practice, such as 30-day returns and the 2-year legal warranty. The documents also include special rules, such as items that cannot be returned and gas canisters that cannot be shipped by express or outside the EU. These let the tests check whether the bot notices exceptions and says “no” when it should.

Orders: 5 invented orders at different stages (shipped, delivered, delayed, processing, returned) answer “Where is my order?”, which the help documents cannot. The table also holds example e-mail addresses, but the bot does not use them: the order number alone is enough, and the e-mail column is never read or shown.

3. Methodology

The chatbot we tested and the one we demo are the same program: both notebooks import `chatbot.py`. `2_run_chatbot.ipynb` runs it as a chat in the notebook or as a Streamlit web chat (localhost:8501) started from one cell; the Groq key is entered with getpass and never stored.

1. **Input guardrails:** before a message reaches the AI, simple patterns hide card numbers, e-mails and phone numbers; known attack phrases get a fixed reply with no AI call; our own tag names are stripped.
2. **Retrieval (RAG):** a multilingual model (e5) turns the question and each document (one short document = one chunk) into meaning vectors, and the 3 closest documents are sent to the AI. For follow-up questions, the previous question is included in the search. For an example question, sending only 3 documents instead of all 25 cuts the document tokens by 86% (316 vs 2,329). Only the last 5 question–answer pairs are sent.
3. **Prompt:** a fixed set of instructions (about 1,000 tokens) tells the AI who it is, to use only the given documents and order data, how to resist tricks, which of the 7 categories and 3 urgency levels to choose, how to write (short, in the language of the customer’s latest message), and shows 3 example answers. The customer’s message is placed inside special tags, so the AI treats it as data, not instructions.
4. **AI model and tool:** answers come from gpt-oss-120b on Groq’s free service, with low randomness (temperature 0.2) and short internal thinking. When a customer gives an order number, the AI can request a lookup; the program checks that the tool exists and runs one fixed, parameterised query by order number only on the read-only database, so the AI can never change data or write its own SQL.
5. **Output guardrails:** the AI must answer in a fixed format (category, urgency, answer, sources), which the program checks before showing it. A final safety filter replaces any reply that contains a card number or the secret canary code hidden in the instructions (a sign of a prompt leak).

4. Evaluation design

We defined 34 test cases in 10 groups in advance and changed them only when the design changed (no e-mail check, no human hand-off). They range from a non-existent product to prompt injection, pasted card details, typos and a German question. Each case defines the expected behaviour (answer or decline), required and forbidden content, tool use, category, urgency, a reference answer and gold documents.

Pass rate (main metric): right behaviour, all required facts, nothing forbidden and a correct tool decision.

Other metrics: Exact Match for category and urgency; valid JSON (all four fields and an allowed category); correct tool decisions; precision@3, recall@3 and MRR for six retrievers (27 cases with a gold document); BLEU and ROUGE-L against the reference answers; tokens and latency per answer.

Two extra checks: a guardrail ablation (attack and off-topic cases rerun with guardrails off) and LLM-as-judge (`gpt-oss-20b`) scoring correctness, groundedness and helpfulness from 1 to 5; a reply passes the judge if correctness is at least 4 and the expected behaviour is followed.

5. Results

The final chatbot passed 33 of 34 cases (97%); the only "failure" (T30) was a grading error (see below).

Measure	Result
Category / urgency exact match	97% / 88%
Valid JSON · correct tool decisions	100% · 100%
LLM judge: correctness · groundedness · helpfulness (1–5)	5.00 · 5.00 · 4.59
Judge agrees with the rule-based pass/fail	33 of 34 cases (97%)
BLEU · ROUGE-L against reference answers	0.33 · 0.61
Per answer: input tokens · output tokens · latency	1,604 · 103 · 8.1 s
Retrieval (e5 vector search): precision@3 · recall@3 · MRR	0.35 · 0.96 · 0.90

Retrieval: e5 had the best MRR of the six retrievers (RRF 0.85, hybrid TF-IDF 0.81, BM25 0.69). Precision@3 is low by design: usually only 1 of the 3 returned documents is relevant (maximum about 0.33).

What the numbers mean

- **The only failure is a scoring error.** In T30 the bot correctly refused pasted card details and pointed to the secure checkout, but "can't process payments" is not in the refusal keyword list, so the rules failed it (automatic diagnosis: "wrong behaviour (refusal)"), while the judge gave it full marks.
- **The guardrail ablation shows the same effect.** With guardrails off, 4 of 6 attack and off-topic cases passed instead of 5, but the extra failure (T28) was again a correct refusal missed by the keyword list. The guardrails' real value: messages with known attack phrases get no AI call, and card numbers, e-mails and phone numbers are masked before anything is sent.
- **The judge found no wrong answers:** all replies scored 5/5 for correctness and groundedness. Helpfulness was lowest (3 of 5) for two refusals (T27, T28) and a very short answer (T10).
- **BLEU and ROUGE measure wording, not truth:** correct but reworded answers scored as low as 0.02 BLEU, so these scores serve only as context.
- **Order lookups are slower:** the four order-number cases (T20–T23) took 16–24 s, 2–3 times the 8.1 s average, because the tool result needs a second model call.

6. Limitations and responsible use

The pass rate does not yet mean the bot is ready for real customers.

- **Small, self-written test set:** each test case equals about 3 percentage points, and safety, hallucination and multi-turn have only one case each. The same team wrote the documents and the tests, so the results are likely too positive.
- **Limited scoring methods:** keyword checks miss correct refusals worded differently (T30), and BLEU/ROUGE only count shared words. The judge is a smaller model from the same family as the chatbot, and its perfect correctness scores suggest it grades too generously.
- **Privacy (GDPR):** the lookup uses only the order number, so anyone who knows one can see that order's status (in T22, a customer who gave a different e-mail still received it). Using real customer data would require a data-processing agreement with the API provider.
- **Prompt injection:** our defences (phrase filter, delimiter tags, output checks, read-only tool) follow OWASP's mitigations, but the filter catches only known wordings and blocked messages stay in the chat history. The main protection is the narrow, read-only tool: an attack can only read order statuses.
- **Guardrail code gaps:** the masking rules are simple regular expressions, so the phone pattern also hides dates such as 2026-10-12, while CVV codes and expiry dates stay visible; our tag names are stripped in a single pass, so a nested tag can still rebuild </customer_message>.
- **Retrieval, tool and memory code:** no similarity threshold, so the 3 closest documents are sent even for off-topic questions; order numbers are not format-checked; the tool loop has no call limit; only the last 5 question–answer pairs are sent; a Groq call that fails after 2 default retries gives an empty reply.

Use of AI tools: AI assistants helped with coding and wording; the group reviewed all code and text and can explain every choice.